Project 2: Solving Eigenvalue Problem with the Jacobi Algorithm

Erica Løken, Mads Mestl and Heine Husdal
Department of Physics, University of Oslo
(Dated: September 24, 2025)

https://github.uio.no/heineeh/FYS3150-Project2

In this project, we study an eigenvalue problem q The physical system consits of a horizontal beam of length L with fixed endpoints. When the compressing force is applied, the beam may buckle into various static shapes. The differential equation describing the situation is

$$\gamma \frac{d^2u(x)}{dx^2} = -Fu(x), \tag{1}$$

where γ is a constant defined by the material properties of the beam and $u(x)$ is the vertical displacement of the beam at horizontal position $x \in [0, 1]$. The force F is apllied at $x = L$ directed towards $x = 0$. Since the beam has fixed endpoints, $u(0) = u(L) = 0$, but the endpoints can rotate, $u'(x) \neq 0$.

We begin by scaling (1) to be dimensionless. Further, the equation is discretized discretize and we obtain an tridiagonal eigenvalue problem

$$\mathbf{A}\vec{v} = \lambda\vec{v}, \tag{2}$$

where $\mathbf{A}$ is tridiagonal and the eigenvectors $\vec{v}$ represent the discretized approximation to the true eigenfunctions that solve Eq. (1). We will solve this problem two different ways by first implementing Armadillos eigensolver and then the Jacobi rotation algorithm to compute the eigenvalues and eigenvectors.

PROBLEM 1

We want to scale Eq. (1) to be dimensionless by introducing the dimensionless variable $\hat{x} \equiv \frac{x}{L}$. The derivative transforms as

$$\frac{d}{dx} = \frac{d\hat{x}}{dx}\frac{d}{d\hat{x}} = \frac{d}{dx}\left[\frac{x}{L}\right]\frac{d}{d\hat{x}} = \frac{1}{L}\frac{d}{d\hat{x}}$$

Inserting this in Eq. (1) yields

$$\frac{\gamma}{L^2}\frac{d^2u(\hat{x})}{d\hat{x}^2} = -Fu(\hat{x}) \implies \frac{d^2u(\hat{x})}{d\hat{x}^2} = -\frac{FL^2}{\gamma}u(\hat{x})$$

By defining a dimensionless parameter $\lambda = \frac{FL^2}{\gamma}$ we obtain the scaled differential equation

$$\frac{d^2u(\hat{x})}{d\hat{x}^2} = -\lambda u(\hat{x}), \tag{3}$$

where $\hat{x} \equiv \frac{x}{L}$ is the dimensionless variable. It is important to mention that this notation is a bit unclear, because $u(x)$ and $u(\hat{x})$ are two different functions. We should have used a more precise notation such as $u_x(x)$ and $u_{\hat{x}}(\hat{x})$, but we will keep the shorthand notation for simplicity.

PROBLEM 2

We want to construct a tridiagonal matrix $\mathbf{A}$ for $N = 6$ and solve the eigenvalue problem $\mathbf{A}\vec{v} = \lambda\vec{v}$ using Armadillos solver `eig_sym`. We will verify that the eigenvalues and eigenvectors from this solver are consistent with the analytical results.
We introduce n discretization steps, such that $N = n - 1$. The step size is

$$h = \frac{1}{n} = \frac{1}{N+1},$$

The matrix is tridiagonal with signature (a, d, a), where

$$a = -\frac{1}{h^2}, \qquad d = \frac{2}{h^2},$$

The analytical eigenvalues of $\mathbf{A}$ are given by

$$\lambda^{(j)} = d + 2a\cos\left(\frac{j\pi}{N+1}\right), \qquad j = 1, \ldots, N \tag{4}$$

with corresponding eigenvectors

$$\vec{v}^{(j)} = \left[\sin\left(\frac{j\pi}{N+1}\right), \sin\left(\frac{2j\pi}{N+1}\right), \ldots, \sin\left(\frac{Nj\pi}{N+1}\right)\right]^T, \qquad j = 1, \ldots, N \tag{5}$$

Before comparison of numerical and analytical results, the eigenvectors will be normalized using Armadillos `normalise`. We compared the results by taking the absolute error of the numerical and analytical eigenvalues $|\lambda_{\text{num}} - \lambda_{\text{ana}}|$ and the absolute error $|v_{i,\text{num}} - v_{i,\text{ana}}|$ of each element v_i in each eigenvector $\vec{v}^{(j)}$.

The results show excellent consistency with the analytical results:

$$\max|\lambda^{(j)}_{\text{num}} - \lambda^{(j)}_{\text{ana}}| = 5.684 \cdot 10^{-14}, \qquad \max|v^{(j)}_{i,\text{num}} - v^{(j)}_{i,\text{ana}}| = 7.772 \cdot 10^{-16}.$$

The code for this problem can be found in the UiO GitHub repository at `code/problem2.cpp` and `code/src/utils.cpp`.

PROBLEM 3

Problem a

The code for this problem can be found in the UiO GitHub repository at `code/src/utils.cpp`.

Problem b

We want to test our function `max_offdiag_symmetric` on the matrix

$$A = \begin{bmatrix} 1 & 0 & 0 & 0.5 \\ 0 & 1 & -0.7 & 0 \\ 0 & -0.7 & 1 & 0 \\ 0.5 & 0 & 0 & 1 \end{bmatrix}$$

The test code finds that the element with the largest absolute value of the matrix is -0.7 at $(k, l) = (1, 2)$.
The code for this problem can be found in the UiO GitHub repository at `code/problem3.cpp`.

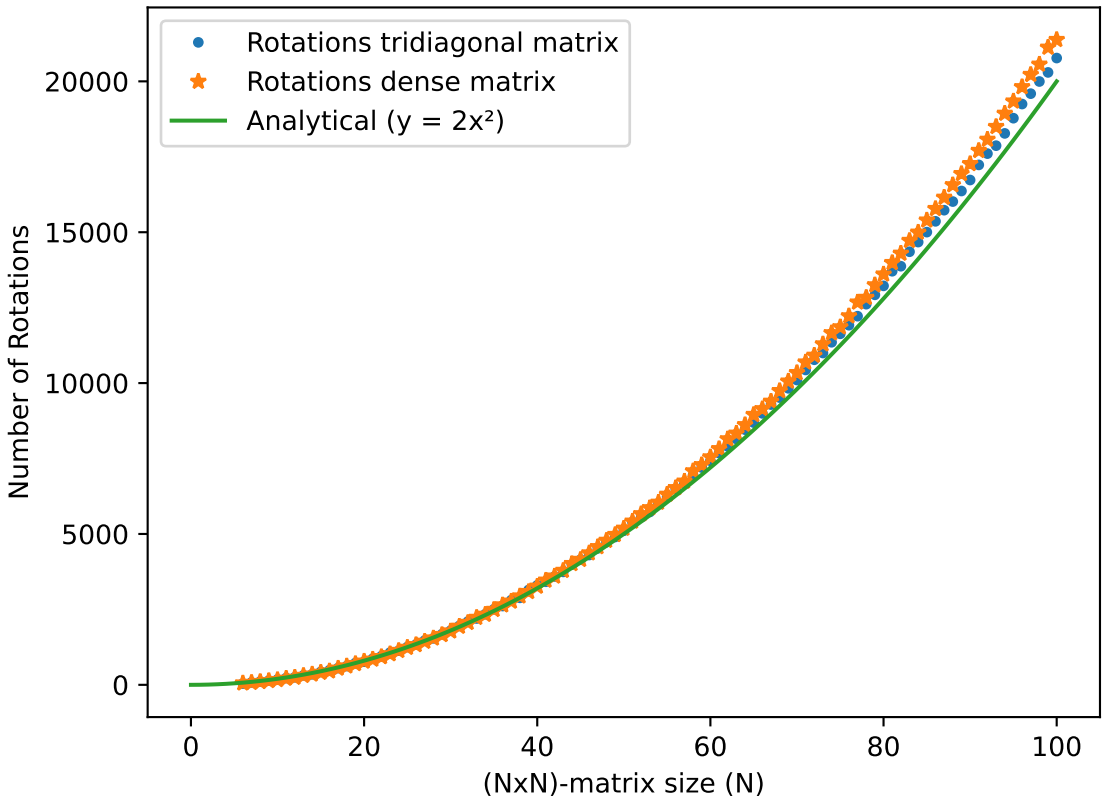

FIG. 1: The number of rotations performed plotted against the matrix size N

PROBLEM 4

Problem a

The code for this problem can be found in the UiO GitHub repository at `code/src/utils.cpp`, and the python plotting script is located at `figs/problem5_plot.py`.

Problem b

We want to test our implementation of the Jacobi rotation method on a matrix $\mathbf{A}$ = tridiag(a,d,a) for $N = 6$ and verify that the eigenvectors and eigenvalues we get from the algorithm agree with the analytical results.
The results from finding the eigenvectors and eigenvalues with the Jacobi rotation algorithm shows great consistency with the analytical results:

$$\max|\lambda^{(j)}_{\text{jacobi}} - \lambda^{(j)}_{\text{ana}}| = 1.989 \cdot 10^{-13}, \qquad \max|v^{(j)}_{i,\text{jacobi}} - v^{(j)}_{i,\text{ana}}| = 5.897 \cdot 10^{-11}.$$

For these results, we used a off-diagonal treshhold value of $\epsilon = 10^{-8}$. The code for this problem can be found in the UiO GitHub repository at `code/problem4.cpp`.

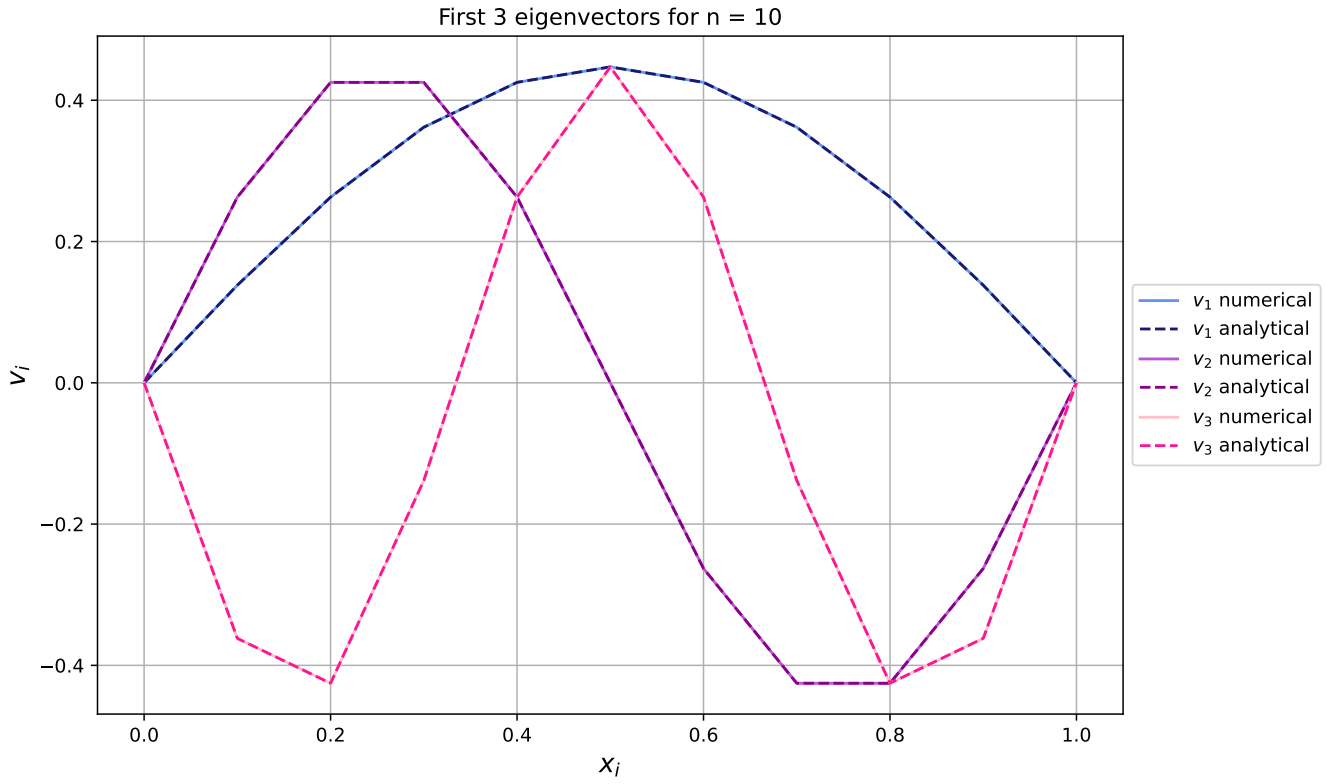

FIG. 2: The three eigenvectors correpsonding to the three lowest eigenvalues for $n = 10$ discretization steps, including boundary points.

PROBLEM 5

Problem a

The code for this problem can be found in the UiO GitHub repository at `code/problem5.cpp`, and the python plotting script is located at `figs/problem5_plot.py`.

Problem b

Figure (1) illustrates the increase in rotations needed for increasing matrix size N. We clearly see there is no notable discrepancy between the between a dense matrix and the specified tridiagonal matrix. Along with the two matrices we have plotted the function $y = 2x^2$, which corresponds surprisingly well with the increase in matrix size. We originally guessed the number of rotations would increase with N^2, since the number of elements increase in this form. The constant 2 is found empirically.

PROBLEM 6

Problem a

We have tested the Jacobi rotation algorithm for $n = 10$ discretization steps, which corresponds to $N = n-1 = 9$ and compared the first three eigenvectors corresponding to the three lowest eigenvalues with the analytical eigenvectors. Figure (2) shows a plot of the first three numerical and analytical eigenvectors for $n = 10$. We see that the numerical eigenvectors matches the analytical eigenvectors very well.

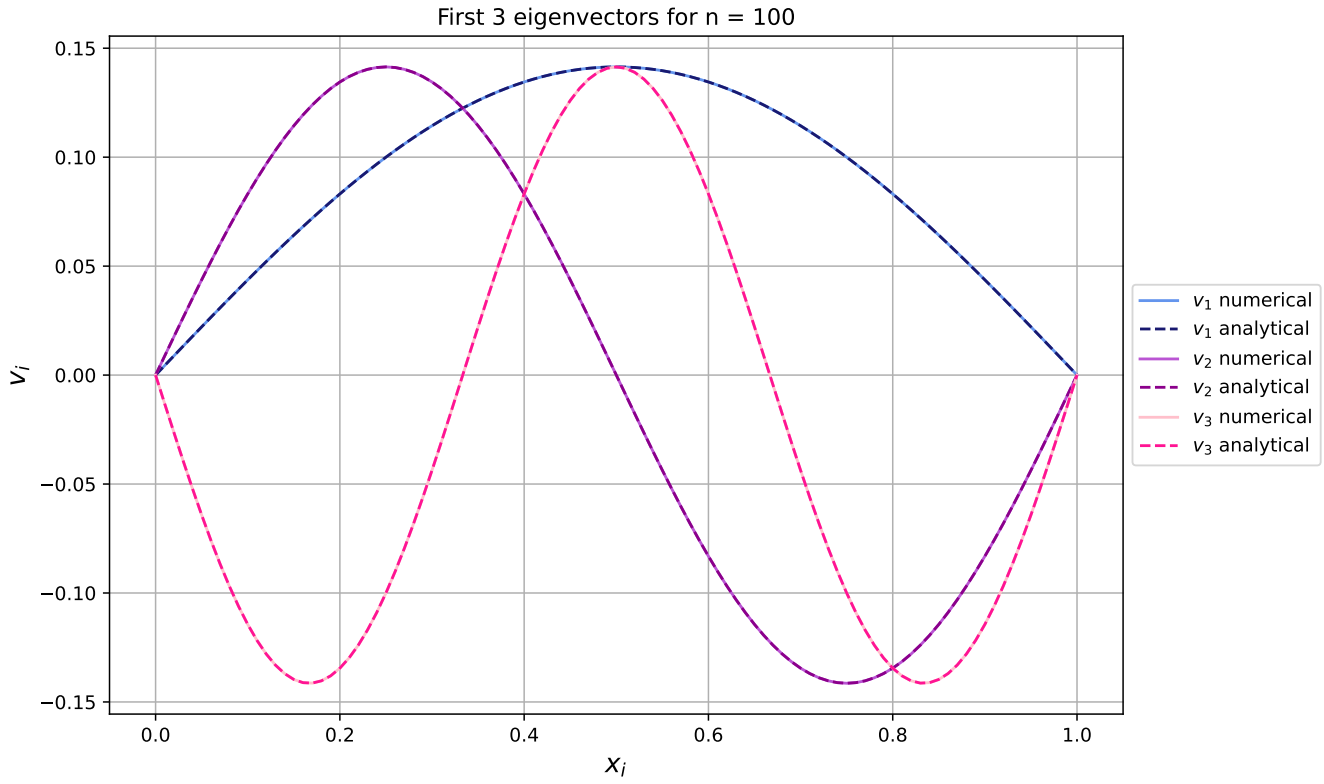

FIG. 3: The three eigenvectors correpsonding to the three lowest eigenvalues for $n = 100$ discretization steps, including boundary points.

The code for this problem can be found in the UiO GitHub repository at `code/problem6.cpp`, and the python plotting script is located at `figs/problem6_plot.py`.

Problem b

Figure (3) shows a plot of the first three eigenvectors for $n = 100$. Again, we see great correspondance between analytical and numeircal eigenvectors.